

Reviewer Pack — Additive Energy Threshold and ACC⁰ Circuit Size

Entry cefe1013b737 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 18:20:51 UTC

Additive Energy Threshold and ACC⁰ Circuit Size

Entry ID: cefe1013b737 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 18:20:51 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For a Boolean function f on n variables, if its additive energy $E(f) \geq n^{2-\varepsilon}$, then the minimal ACC⁰ circuit size of f is at least $\Omega(n^{1+\varepsilon})$ for any $\varepsilon > 0$.

Rationale (proposer's reasoning):

High additive energy implies the function is structured in a way that resists efficient computation by ACC⁰ circuits, which are limited in their ability to handle structured functions.

Taxonomy category: ACC_SIPSER (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f26d42997fb5e326

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def additivity(f):
        n = int(math.log2(len(f)))
        count = 0
        for i in range(n):
            for j in range(i+1, n):
                for k in range(j+1, n):
                    if f[i] + f[j] == f[k]:
                        count += 1
        return count

    def additive_energy(f):
        n = int(math.log2(len(f)))
        return additivity(f) / (n * (n - 1) * (n - 2))

    def acc0_circuit_size_bound(n, epsilon):
        return n ** (1 + epsilon)

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    energy = additive_energy(f)
    epsilon = 0.1
    conjecture_holds = energy >= n ** (2 - epsilon) and acc0_circuit_size_bound(n, epsilon) <= n ** (2 - epsilon)

    return {
        "metric_name": "additive_energy",
        "metric_value": energy,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Energy {energy} < n^(2-{epsilon}) for n={n}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_energy = sum(r["metric_value"] for r in results) / len(results)
std_energy = math.sqrt(sum((r["metric_value"] - mean_energy) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_energy} std={std_energy} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and debug crash logs

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	147559
2	preregistration	ollama_remote	qwen3:8b	0	0	56821
3	novelty	ollama_remote	qwen3:8b	0	0	53274
4	novelty	ollama_remote	qwen3:8b	0	0	41974
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36806
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7165
7	judge	ollama_remote	qwen3:8b	0	0	63496

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 407095 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/cefe1013b737.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cefe1013b737.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cefe1013b737.tar.gz (if generated)